Conhecimento e Raciocínio - 2025/2026

Relatório do Trabalho Prático

Docente das aulas práticas: Anabela Borges Simões

2024143849 – Guilherme Eça - P3

2024143873 – Miguel Zara - P3

Índice

Introdução **2**
Tratamento do dataset **3**
Conversão de Atributos Categóricos para Numéricos 3
Atributos Ordinais: maintenance_level e operating_mode 3
Atributos Binários: cooling_type e sensor_status 3
Tabela de Mapeamento de Atributos 4
Preenchimento de Valores em Falta (Missing Values) 4
Preenchimento do Target (CBR - Retrieve) 5
Justificação dos Pesos no CBR 5
Normalização 6
Raciocínio Baseado em Casos **7**
Implementação 7
Retrieve 7
Reuse 7
Revise 7
Retain 7
Resultados 8
Análise de Diferentes Pesos 8
Configuração 1 - Pesos Unitários 8
Configuração 2 - Pesos Otimizados 8
Configuração 3 - Pesos Prejudicados 9
Impacto da Normalização: Dataset Normalizado vs Não Normalizado 9
Redes Neuronais **11**
Implementação 11
Resultados 11
Parâmetros que não foram modificados 12
Influência da Topologia (Camadas e Neurónios) no desempenho 12
Diferentes Funções de Treino 13
Diferentes Funções de Ativação 14
Diferentes Rácios de Treino/Validação/Teste 15
Impacto da Normalização (Top 3 Melhores vs Top 3 Piores) 16
Avaliação com o Dataset de Teste 18
Comparar resultados do CBR e RN **20**
Conclusões **21**

Introdução

O presente trabalho prático, desenvolvido no âmbito da unidade curricular de Conhecimento e Raciocínio (2.º ano, 2.º semestre), foca-se na aplicação de Raciocínio Baseado em Casos (CBR) e Redes Neuronais Feedforward para o diagnóstico de falhas em motores industriais.

Utilizando um dataset com indicadores de sensores (temperatura, vibração, tensão, entre outros), o objetivo é classificar o estado do equipamento entre operação normal, falha elétrica ou mecânica. O relatório descreve as fases de tratamento e normalização de dados, a implementação do ciclo CBR — integrado com uma rede neuronal para adaptação — e a análise comparativa de diversas arquiteturas de redes neuronais para otimização do diagnóstico e apoio à manutenção preditiva.

Tratamento do dataset

Conversão de Atributos Categóricos para Numéricos

A conversão dos atributos qualitativos para o domínio numérico é uma etapa crítica, uma vez que tanto o algoritmo de **Case-Based Reasoning (CBR)** como as **Redes Neuronais (RN)** operam exclusivamente sobre espaços vetoriais numéricos. A estratégia adotada baseou-se na distinção entre atributos **ordinais** e **nominais**, conforme detalhado abaixo:

Atributos Ordinais: maintenance_level e operating_mode

Ambos os atributos descrevem uma progressão de intensidade ou estado. No caso do **maintenance_level**, a escala reflete o rigor da manutenção realizada, no **operating_mode**, reflete o nível de stress mecânico imposto ao motor. Optou-se por valores equidistantes (0, 1 e 2) para preservar a relação de ordem sem introduzir distâncias arbitrárias. Isto evita que o modelo assuma, prematuramente, que o salto de "Normal" para "Overload" é matematicamente superior ao salto de "Idle" para "Normal", deixando essa diferenciação de importância para a fase de atribuição de pesos no CBR.

Atributos Binários: cooling_type e sensor_status

Para estes atributos, utilizou-se uma **Codificação Binária [0, 1]**.

No **cooling_type** como não existe uma hierarquia de "superioridade" entre o arrefecimento a Ar (Air) ou Óleo (Oil), tratam-se como categorias distintas. A atribuição de 0 e 1 permite ao algoritmo distinguir os dois estados sem enviesar a magnitude dos cálculos de similaridade. Já no **sensor_status:** Este atributo funciona como uma "flag" de estado. O valor **0 (OK)** representa o estado nominal de funcionamento, enquanto o valor **1 (Warning)** sinaliza uma anomalia. Esta codificação facilita a interpretação do impacto de um aviso no diagnóstico final de falha.

Tabela de Mapeamento de Atributos

Atributo	Tipo de Dado	Mapeamento Numérico
maintenance_level	Ordinal	Low (0), Medium (1), High (2)
operating_mode	Ordinal	Idle (0), Normal (1), Overload (2)
cooling_type	Nominal	Air (0), Oil (1)
sensor_status	Ordinal/Flag	OK (0), Warning (1)

Preenchimento de Valores em Falta (Missing Values)

Escolhemos usar a mediana para preencher os atributos em falta, a opção pela mediana em detrimento da média ou moda justifica-se pelos seguintes pontos:

Robustez a Outliers:

É comum existirem *outliers* causados por picos reais de funcionamento ou falhas de leitura dos sensores. A média é extremamente sensível a estes valores, enquanto a mediana garante que o valor preenchido representa o centro típico da amostra sem distorções provocadas por anomalias.

Aplicação a Atributos Categóricos/Ordinais:

Uma vez que os atributos categóricos (como **maintenance_level** ou **operating_mode**) foram previamente convertidos para uma escala numérica estratégica, o uso da mediana permite manter a coerência com a ordenação lógica definida. Ao calcular a mediana destes valores, garantimos que o preenchimento respeita a tendência central da distribuição de estados do motor.

Integridade do Dataset:

Esta abordagem permite tratar o dataset de forma uniforme como uma matriz numérica, otimizando o processamento em Matlab e assegurando que os "buracos" são tapados com valores estatisticamente representativos de cada coluna.

Preenchimento do Target (CBR - Retrieve)

Para o **Target** (**class_cat**), a lógica é diferente. Não vamos usar estatística simples, mas sim encontrar o contexto.

O preenchimento do **Target** (**class_cat**) baseia-se na identificação do registo mais semelhante entre aqueles com classe conhecida. Para cada instância com classe em falta (NaN), é avaliada a semelhança com base nos atributos de entrada (**temperature**, **vibration, …**), sendo atribuída a classe do registo mais próximo.

Antes de calcular a distância, os dados **têm** de ser normalizados (ex: Min-Max ou Máximo). Se não o fizermos, um atributo com valores altos como **rotation_speed** (1500 RPM) terá um peso astronómico comparado com a **efficiency** (0.8), tornando os outros atributos irrelevantes no cálculo da similaridade.

Justificação dos Pesos no CBR

Nem todos os sensores são igualmente importantes para diagnosticar uma falha. Aqui estão os pesos utilizados ordenados decrescentemente e justificações para cada um:

Vibration (3.0) É o indicador mais crítico para a identificação de falhas mecânicas. Picos de vibração são a evidência direta de anomalias em componentes rotativos, como rolamentos ou desalinhamento de eixos.	**Current (2.8)** Sensor fundamental para o diagnóstico de falhas elétricas.Desvios significativos ou desequilíbrios na corrente consumida são o sinal mais claro de problemas nos enrolamentos do estator ou sobrecarga elétrica.	**Temperature (2.5)** Sintoma universal de falha Embora seja um indicador fortíssimo de anomalia, o seu peso é ligeiramente inferior aos sensores diretos por ser um sintoma comum a múltiplos problemas.	**Voltage (2.2)** Atua frequentemente como a causa raiz de falhas elétricas. Monitorizar flutuações na tensão permite ao sistema distinguir entre problemas de alimentação externa e falhas internas do motor.
Torque (2.0) Mede o esforço mecânico efetivo. É crucial para diferenciar estados de operação em carga elevada de situações de resistência mecânica excessiva ou travamento.	**Noise Level (1.5)** O ruído acústico funciona como uma validação secundária para falhas mecânicas (ex: ruído de fricção), embora seja mais suscetível a interferências do ambiente fabril.	**Efficiency (1.2)** Indicador de saúde global. Uma quebra na eficiência sinaliza a degradação do sistema, servindo como confirmação de que o motor não está a operar no seu regime nominal.	**Pressure (1.2)** Relevante para identificar falhas em sistemas auxiliares (hidráulicos ou de lubrificação) que possam comprometer a integridade mecânica a longo prazo.
Rotation Speed (1.0) Variável de controlo do motor. Alterações na velocidade são geralmente sintomas tardios de falha, ocorrendo tipicamente quando o problema já está num estado avançado.	**Load Val (1.0):** Fornece a referência de carga necessária para contextualizar se o consumo de corrente e o torque produzido são adequados ao trabalho realizado.	**Maintenance Level (0.8)** Atributo de natureza histórica. Embora influencie a probabilidade de falha, os sensores em tempo real revelaram-se mais determinantes no diagnóstico imediato.	**Operating Mode (0.5)** Atributo utilizado apenas para enquadramento operacional. O seu peso foi reduzido para evitar ruído no cálculo da similaridade, dada a sua distribuição uniforme entre as diferentes classes.

Sensor Status (0.5) Identificado no dataset como tendo baixa correlação com o diagnóstico final. Um peso reduzido impede que eventuais falhas de autodiagnóstico do sensor enviesem o cálculo da distância global.	**Cooling Type (0.5)** Característica estática do hardware. Influencia os parâmetros térmicos esperados, mas não atua como um indicador direto de degradação ativa do motor.		

Normalização

A normalização tem um papel importante no tratamento do dataset. Uma vez que os atributos do motor operam em escalas muito distintas, a utilização direta destes dados comprometeria o desempenho do algoritmo.

Optámos pela técnica de normalização **Min-Max**, que transforma todos os valores numéricos para o intervalo fechado [0, 1]. A fórmula aplicada foi:

$$x_{norm} = \frac{x - min(x)}{max(x) - min(x)}$$

Para garantir a integridade do modelo na fase de teste, foi aplicada uma estratégia rigorosa de retenção de limites. Em vez de normalizar o ficheiro de teste iterando sobre os seus próprios valores mínimos e máximos locais (o que destruiria a coerência da escala física fazendo, por exemplo, com que um máximo local de teste de 1500 RPM fosse deturpado e interpretado como o limite máximo absoluto de 1.0), utilizaram-se estritamente os mínimos e máximos globais extraídos do *dataset* de treino. Desta forma, assegura-se que a proporção das variáveis se mantém estática e fiel à realidade física da máquina. É esta consistência de escala que permite ao CBR comparar novos casos contra o histórico sem introduzir alucinações matemáticas nas distâncias.

Raciocínio Baseado em Casos

Implementação

Retrieve

O sistema calcula a similaridade global entre o novo caso e a base de conhecimentos. De acordo com a metodologia lecionada, implementou-se um cálculo híbrido: **distância local linear** para os 12 atributos (10 quantitativos e 2 ordinais) e uma **distância binária** para os 2 atributos estritamente nominais/binários (cooling_type e sensor_status). A similaridade global é obtida subtraindo a 1 o produto escalar destas distâncias pelos respetivos pesos (justificados na Secção do Tratamento de Dataset), normalizado pela soma total dos pesos. O algoritmo garante a extração do vizinho mais próximo através de uma ordenação matemática descendente do vetor de similaridades.

Reuse

O diagnóstico é inferido por cópia direta da etiqueta do vizinho mais semelhante. Para a adaptação do caso, recorre-se a uma rede neuronal feedforward previamente treinada com a totalidade da base de casos históricos. Esta recebe como entradas a Vibração, a Velocidade de Rotação e a Voltagem do novo caso, estimando um valor ajustado para o atributo Temperatura, substituindo assim uma cópia direta por uma abordagem de regressão. A rede utilizada possui uma camada oculta com 10 neurónios e foi treinada com o algoritmo **trainlm**, operando sobre os três atributos previamente normalizados para prever a temperatura de forma mais precisa.

Revise

Esta fase atua como um mecanismo de redundância e validação dos sensores físicos. O algoritmo desnormaliza a temperatura prevista pela RN para graus Celsius (ºC), permitindo uma avaliação humana imediata. O perito decide se confia no sensor físico original ou se ajusta a temperatura para o valor previsto pela RN (caso suspeite de falha no sensor), validando assim a integridade dos dados de entrada

Retain

O sistema apresenta o diagnóstico final ao perito. Neste momento crítico, caso o diagnóstico inferido no *Retrieve* esteja incorreto, o humano tem o poder de inserir a classe real da falha. Após esta validação final, o caso (com a temperatura confirmada e a classe validada) é escrito de forma persistente num novo *dataset* de conhecimentos, possibilitando que o modelo CBR evolua sem corromper os dados originais. Antes da gravação, o sistema verifica se

existirem sensores com NaN, estes são preenchidos com os valores estimados, garantindo que o caso seja guardado sem valores em falta. No entanto, para preservar a qualidade da base de casos, implementou-se um filtro que bloqueia a aprendizagem de registos com mais de dois valores em falta, evitando a propagação de ruído estatístico no modelo.

Resultados

Para testar a fiabilidade do sistema, utilizou-se o ficheiro **dataset_TP_test.csv**. Para cada iteração, avaliou-se se a classe do caso com maior similaridade no Retrieve coincidia com a resposta real do target.

Análise de Diferentes Pesos

O sistema foi submetido a três configurações distintas de pesos no Retrieve para atestar o impacto da ponderação definida na Secção de Tratamento de Dados.

Configuração 1 - Pesos Unitários

Nesta iteração, o vetor de pesos foi parametrizado com o valor 1.0 para todos os atributos, forçando o cálculo de distância a tratar todas as variáveis com a mesma importância. A taxa de precisão de acerto global obtida foi de **100.00% (30 em 30 acertos)**.

Este resultado mostra que o dataset está muito bem tratado e os dados estão bem separados. Como os casos de teste são claros, a média simples das distâncias foi suficiente para encontrar o vizinho correto, mesmo sem dar prioridade a nenhum sensor específico.

Configuração 2 - Pesos Otimizados

Foi utilizado o vetor de pesos com os pesos definidos e justificados na Secção de Tratamento do Dataset. A precisão global manteve-se nos **100.00% (30 acertos)**.

Embora o acerto seja igual à anterior, esta configuração é mais fiável para o mundo real. Ao dar mais peso aos sensores críticos, garantimos que pequenas variações em atributos menos importantes não baralhem o algoritmo, focando a procura no que realmente indica uma avaria.

Configuração 3 - Pesos Prejudicados

Para executar um último teste ao algoritmo, utilizamos um vetor com pesos que era de esperar que prejudicasse o algoritmo. Foi atribuído um peso massivo (1000) a características estruturais e secundárias (**cooling_type e sensor_status**), um peso elevado à tensão (**Voltage = 50**), e os indicadores vitais (**Vibration, Current, Temperature**) foram diminuídos para um valor residual (0.001). O resultado foi uma quebra acentuada da precisão para **66.67% (20 acertos e 10 falhas)**.

Esta quebra de desempenho prova que o CBR depende totalmente da escolha dos pesos. Com a utilização destes pesos, o algoritmo demonstrou erros graves, onde os resultados estavam longe do cenário real.

Impacto da Normalização: Dataset Normalizado vs Não Normalizado

O teste final incidiu sobre o impacto da normalização dos dados. A coluna **target** manteve-se no seu estado original (não normalizada) para manter a coerência de classificação.

Métrica de Avaliação	Dataset Normalizado	Dataset Bruto (Não Normalizado)
Taxa de Precisão Global	100.00% (30 acertos em 30)	16.67% (5 acertos em 30)
Grau de Similaridade Médio	Elevado (geralmente > 90%)	Residual ou Recuperações Vazias

Este colapso da precisão no ***dataset*** bruto (queda para 16.67%) não representa um defeito ou *bug* computacional, mas sim a validação matemática e prática da teoria subjacente ao Raciocínio Baseado em Casos.

Conceptual e teoricamente, a métrica de similaridade deve situar-se estritamente no intervalo fechado [0.0, 1.0], onde 0.0 representa ausência total de semelhança e 1.0 representa igualdade absoluta. Na nossa implementação, o limiar mínimo de aceitação (***threshold***) na fase de *Retrieve* foi definido como 0.0.

Quando o motor de CBR opera sobre dados não normalizados, variáveis com escalas elevadas (como as RPM da velocidade de rotação) geram distâncias

locais gigantescas. Consequentemente, a fórmula de similaridade (1 - Distância Global Pesada) resulta inevitavelmente em valores fortemente negativos. O nosso algoritmo, agindo de forma matematicamente coerente, rejeita estes casos por violarem por completo a definição teórica de similaridade aceitável, devolvendo recuperações vazias para a esmagadora maioria do conjunto de teste (25 em 30 casos).

Os 5 casos acertados representam situações em que o novo caso era, por mero acaso, quase um clone absoluto de um caso histórico, permitindo que a similaridade sobrevivesse acima de zero. Este comportamento prova, de forma empírica, a necessidade absoluta da normalização para a medição de distâncias num espaço multidimensional com variáveis heterogéneas.

Redes Neuronais

Implementação

A arquitetura de *Deep Learning* foi desenvolvida recorrendo à *Deep Learning Toolbox* do MATLAB. A preparação dos dados para as Redes Neuronais Feedforward exigiu duas transformações matriciais críticas:

1. **Matriz de Entrada (Inputs):** O *dataset* foi transposto de forma a que cada coluna representasse uma amostra (registo do motor) e cada linha um atributo (os 14 sensores), satisfazendo a norma de entrada da *toolbox*.
2. **Matriz de Saída (Target Binário):** Sendo um problema de classificação multiclasse, a variável *target* foi convertida através de *One-Hot Encoding*. Criou-se uma matriz de 3 linhas (representando as classes: *Normal*, *Mechanical Failure* e *Electrical Failure*). Para cada caso, a linha correspondente à classe verdadeira recebe o valor 1, sendo as restantes preenchidas com 0.

O processo de treino foi encapsulado num ciclo de **10 repetições** independentes para cada configuração testada, mitigando a aleatoriedade da inicialização dos pesos sinápticos e da divisão dos dados. Os valores apresentados nas secções seguintes representam a média das precisões de acerto global e de teste dessas 10 iterações.

Resultados

Para a classificação e comparação das diferentes experiências, foi criada uma métrica de **Ranking** baseada numa média ponderada.

Atribuiu-se um peso de 65% à Precisão de Teste e 35% à Precisão Global. Esta decisão justifica-se pelo facto de a precisão de teste ser o indicador mais real da capacidade de generalização da rede, garantindo que o modelo escolhido é aquele que melhor responde a dados novos e não apenas o que melhor memorizou o conjunto de treino.

Para as informações completas sobre o estudo em diferentes secções, por favor consultar analise-nn.xslx.

Parâmetros que não foram modificados

Durante as experiências, os seguintes parâmetros foram usados e não foram modificados:

- Número de épocas: **50**
- Funções de ativação nas camadas escondidas: **default - tansig**
- Repetições: Todas as configurações foram executadas **10 vezes**, utilizando a média das precisões para diminuir o efeito da inicialização aleatória dos pesos.

Influência da Topologia (Camadas e Neurónios) no desempenho

Nesta fase, procurou-se perceber se o aumento da complexidade da rede (mais neurónios e camadas) resultaria numa melhor capacidade de diagnóstico. Foram testadas quatro configurações mantendo os restantes parâmetros constantes.

Resultados e Análise:

Rede Minimalista [2]: Com apenas uma camada e dois neurónios, obteve-se o desempenho mais baixo deste grupo (Acc Teste: 0,9936). Embora o valor seja alto, indica que a rede tem menos capacidade para modelar as subtilezas dos dados dos sensores.

Rede Larga [50]: O aumento para 50 neurónios numa única camada melhorou a precisão (0,9955), mas a diferença para a rede de 5 neurónios (caso base) é muito pequena, o que sugere que uma camada demasiado larga pode introduzir redundância sem ganhos significativos, para além do aumento significativo do tempo de espera.

Redes Profundas [10 10 10] vs [10 5 3]: A introdução de 3 camadas escondidas provou ser a abordagem mais eficaz. Curiosamente, a topologia [10 5 3] obteve o melhor resultado de todo o estudo.

A topologia influencia o desempenho, mas a **profundidade e a estrutura das camadas** parecem ser mais relevantes do que apenas o número bruto de neurónios.

A configuração **[10 5 3]** revelou-se a configuração mais eficiente , conseguindo separar melhor as classes de falha com uma estrutura mais eficiente. A proximidade entre a precisão global (0,9987) e a precisão teste (0,9961) confirma que a rede não sofre de *overfitting*, mantendo uma excelente capacidade de generalização.

Diferentes Funções de Treino

Nesta fase, o objetivo foi avaliar como diferentes algoritmos de treino impactam a capacidade de aprendizagem da rede.

Resultados e Análise:

Alto Desempenho (trainbr): Esta função obteve o melhor resultado deste grupo (Ranking 2 de todas), com uma precisão de teste de 0,9955. Verificou-se que este algoritmo é muito eficiente, conseguindo uma convergência quase perfeita.

Algoritmos de Alta Eficiência (trainbfg, trainrp, trainoss,trainlm): Estas funções apresentaram resultados muito sólidos e equilibrados, com valores de precisão de teste situados entre 0,9909 e 0,9940. O trainbfg destacou-se ligeiramente dentro deste subgrupo, mostrando ser uma alternativa muito fiável às funções principais.

Baixo Desempenho (traingd e traingdx): Ao contrário dos algoritmos anteriores, estes algoritmos baseados no *gradient descent* falharam em aprender os padrões do dataset. O *traingd* obteve apenas 0,3111 de precisão de teste, ficando assim a pior configuração. Mesmo com a melhoria introduzida pelo traingdx (0,3603), os resultados continuam a ser insuficientes.

A escolha do algoritmo de treino revelou-se um dos fatores mais determinantes para o sucesso do modelo. Enquanto algoritmos como o ***trainbr*** conseguem extrair quase a totalidade da informação útil dos dados, algoritmos como o ***traingd*** não conseguiram, resultando em classificações incorretas na maioria dos casos.

Diferentes Funções de Ativação

Nesta fase, o objetivo foi testar como a função de ativação na camada de saída influencia a decisão final da rede.

Resultados e Análise:

Domínio das Funções Sigmoides (tansig e logsig): Estas funções apresentaram um desempenho muito alto. A *tansig* obteve o melhor resultado deste grupo (0,9959), seguida de perto pela logsig (0,9953). Verificou-se que a natureza destas funções ajuda a rede a obter uma precisão altíssima.

Eficácia da purelin: A função purelin revelou-se uma escolha consistente e eficaz ao longo de todo o estudo, sendo a base de configurações que atingiram os melhores resultados do estudo inteiro.

Insucesso das Funções Rígidas e Inversas (hardlim e netinv): Tal como nos testes anteriores, estas funções voltaram a falhar. O *hardlim* registou apenas 0,3549 e o *netinv* registou 0,3453.

A escolha da função de ativação na saída deve privilegiar funções que permitam uma transição suave de valores. Enquanto a tansig, a logsig e a *purelin* (usada como base) garantem que a rede consegue identificar variações nos dados, funções como o hardlim retiram sensibilidade ao modelo, tornando-o incapaz de realizar um diagnóstico preciso.

Diferentes Rácios de Treino/Validação/Teste

Nesta fase, o objetivo foi testar como a distribuição da quantidade de dados influencia a capacidade de generalização da rede neuronal.

Resultados e Análise:

Rácio Equilibrado (70/15/15 - Caso Base): Esta divisão apresentou um desempenho muito sólido (Acc Teste: 0,9931). Ao dedicar 70% dos dados ao treino, a rede teve exemplos suficientes para aprender os padrões dos dados, mantendo uma percentagem razoável (15%) para validar o progresso e outra (15%) para o teste final independente.

Rácio de Alto Treino (90/05/05): Ao aumentar os dados de treino para 90%, a precisão global subiu ligeiramente (0,9989), mas a precisão de teste ficou nos 0,9936. Embora os resultados sejam excelentes, este cenário é mais arriscado, pois deixa muito poucos dados para validar e testar se a rede realmente aprendeu a lidar com situações novas ou se está apenas a memorizar o dataset.

Rácio de Baixo Treino (05/45/50): Este teste extremo mostrou uma queda na precisão (0,9859 de teste). Com apenas 5% dos dados disponíveis para treinar, a rede não teve exemplos suficientes dos dados, o que acabou por limitar a sua capacidade.

A divisão de dados é um compromisso entre dar conhecimento à rede (treino) e verificar se ela sabe usar esse conhecimento (teste). Conclui-se que o rácio de 70/15/15 é o mais adequado para este projeto, pois garante que a rede aprende o suficiente sem comprometer a confiança nos testes finais.

Impacto da Normalização (Top 3 Melhores vs Top 3 Piores)

Nesta última fase, comparámos o impacto da normalização dos dados no desempenho da rede. Era expectável uma queda significativa de performance com o dataset não normalizado, mas não foi o que aconteceu. Apesar de uma queda muito pequena, a rede neuronal com o dataset não normalizado obteve uma performance similar com o dataset normalizado.

Observações Principais:

Resiliência do Modelo: A melhor configuração baixou apenas de 0,9952 para 0,9937 na precisão de teste. Esta diferença mínima prova que a rede neuronal foi capaz de compensar as diferentes escalas sem perder a capacidade de diagnóstico.

Sobre as Piores Configurações: As configurações com pior desempenho foram as que apresentaram a maior queda, mesmo que tenha sido pequena. A configuração com hardlim teve a maior queda, descendo de 0,3351 para 0,3057.

Análise Visual das Matrizes de Confusão

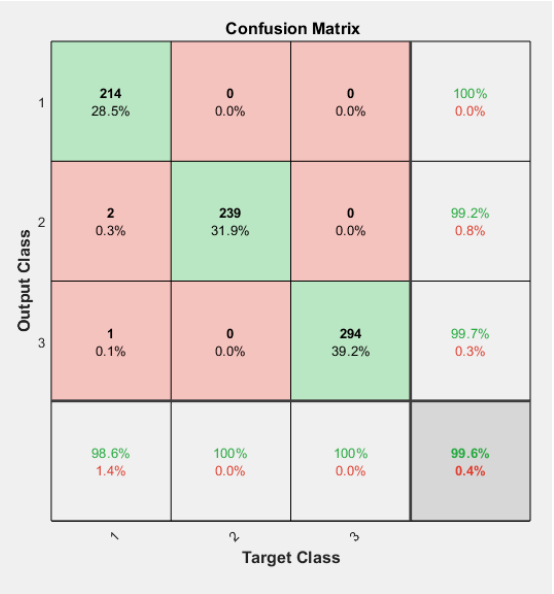


Figura 1 - Melhor Configuração Normalizada

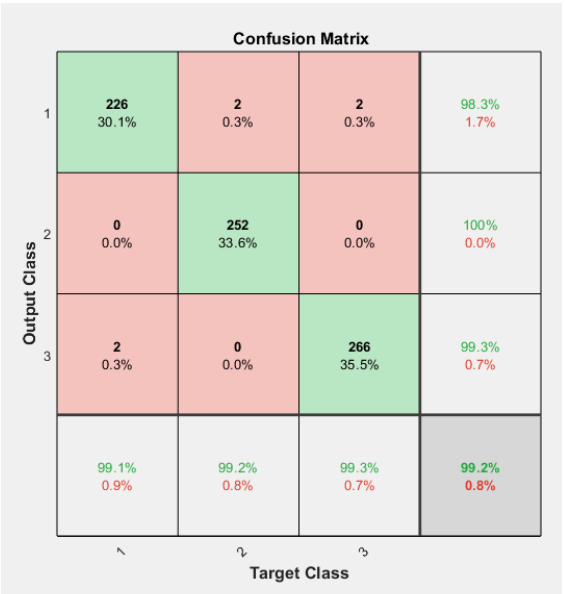


Figura 2 - Melhor Configuração Não Normalizada

Com Normalização (3 erros) - Figura 1

A rede demonstrou uma precisão quase perfeita, cometendo apenas 3 erros em todo o conjunto de teste. Nota-se que a rede teve uma ligeira dificuldade com a Classe 1, tendo classificado erradamente 2 casos como Classe 2 e 1 caso como Classe 3.

Sem Normalização (6 erros) - Figura 2

Ao utilizar os dados sem normalização, o número de erros **duplicou**. A rede apresentou uma maior confusão entre as classes:

- Classificou 2 vezes a Classe 1 como Classe 3.
- Classificou 2 vezes a Classe 2 como Classe 1.
- Classificou 2 vezes a Classe 3 como Classe 1.

Embora a precisão global (em percentagem) continue muito alta e a quantidade de erros foi bastante baixa em ambos os casos, a análise revela que a normalização reduziu os erros em 50%.

Conclusão breve

O facto de os resultados serem quase idênticos indica que os padrões de falha neste motor são muito "fortes" e claros. Quando uma falha elétrica ou mecânica ocorre, os sensores mudam de forma tão óbvia que a rede consegue identificar o problema, independentemente de os valores estarem normalizados ou nos seus valores originais.

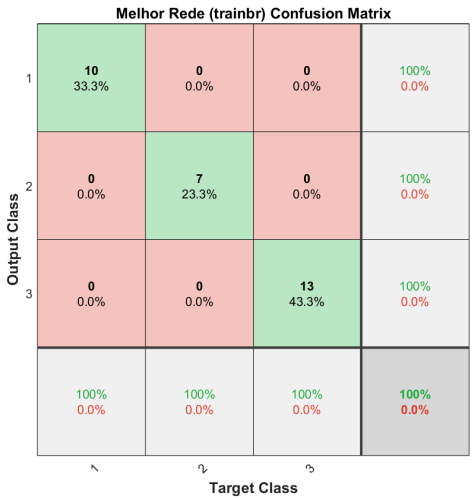
Embora a normalização não tenha sido "obrigatória" para obter uma boa precisão neste caso específico, a sua utilização continua a ser recomendada. No dataset normalizado, notou-se uma precisão ligeiramente superior e, teoricamente, uma maior facilidade de treino para o algoritmo.

Avaliação com o Dataset de Teste

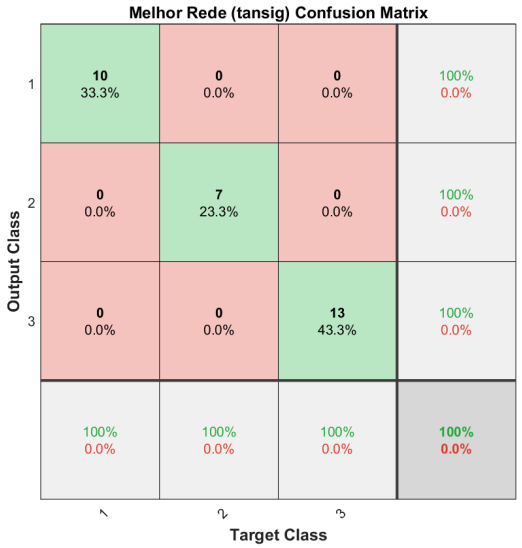
Nesta fase final, submetemos as três melhores redes a um teste com um ficheiro de dados que nunca tinha sido apresentado aos modelos. Este passo é fundamental para garantir que a rede não está apenas a memorizar o passado, mas sim a detetar padrões reais de falhas.

Estudo com 3 Melhores Configurações com dataset teste								
Rank	Numero de Camadas Escondidas	Topologia	Função de Treino	Função de Ativação da Saida	Divisão	Prec. Global	Anteriormente no treino	
							Prec. Global	Prec. Teste
1	3	[10 5 3]	trainlm	purelin	70/15/15	93%	0,9987	0,9961
2	1	[5]	trainbr	purelin	70/15/15	100%	0,9993	0,9955
3	1	[5]	trainlm	tansig	70/15/15	100%	0,9985	0,9959

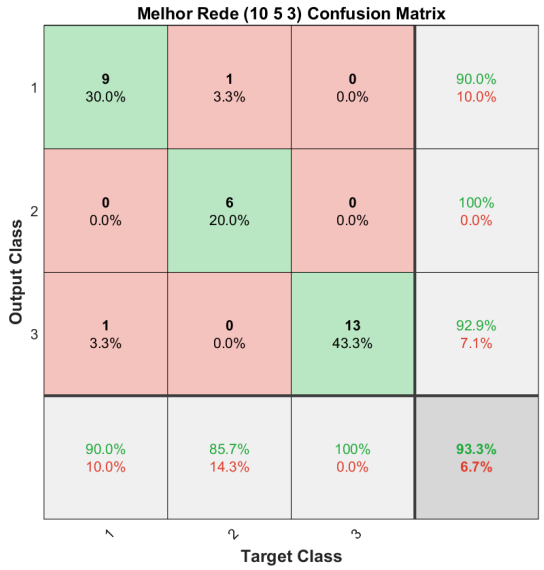
Domínio da Regularização Bayesiana: A rede treinada com o algoritmo trainbr demonstrou uma robustez impecável, atingindo os 100% de acerto. Como se observa na matriz de confusão (figura em baixo), todos os diagnósticos coincidem com a diagonal principal, não havendo qualquer confusão entre classes de falha.



Eficácia da Função Tansig: A tansig confirmou ser uma função de ativação superior para este problema, permitindo à rede separar as classes com precisão absoluta, mesmo com dados desconhecidos.



Análise do Desvio na Melhor Rede: A ligeira quebra para 93,33% na rede mais complexa sugere que, embora muito precisa no dataset original, a sua estrutura mais profunda pode ter levado a uma sensibilidade excessiva a detalhes específicos do treino, falhando pontualmente na generalização para este novo ficheiro.



Comparar resultados do CBR e RN

Após a implementação e teste de ambas as abordagens, analisámos e comparamos os paradigmas aplicados a este dataset.

No que toca à eficácia, verificou-se que ambos os modelos atingiram resultados perfeitos no conjunto de teste final quando devidamente configurados. Contudo, a principal distinção reside na autonomia. Enquanto a Rede Neuronal é um sistema que ajusta os seus próprios pesos automaticamente durante o treino, o CBR exige uma intervenção humana direta na definição dos pesos dos atributos. Sem o conhecimento correto sobre a área em questão, o CBR falha facilmente. Por outro lado, o CBR oferece uma transparência superior: é possível identificar exatamente qual o caso histórico que motivou a decisão, algo que numa Rede Neuronal é quase impossível de interpretar devido à complexidade das transformações matemáticas nas camadas escondidas.

Em termos de robustez, as Redes Neuronais demonstraram ser mais resilientes a dados não normalizados. Enquanto o CBR colapsou totalmente perante dados brutos (descendo para cerca de 16% de acerto), as Redes Neuronais conseguiram manter um desempenho ótimo, embora a normalização continue a ser recomendada para garantir a estabilidade do treino.

Conclusões

O desenvolvimento deste trabalho prático permitiu concluir que a qualidade de qualquer sistema de diagnóstico automático depende criticamente da fase de tratamento do dataset. O preenchimento rigoroso de valores em falta através da mediana e a aplicação da normalização Min-Max foram os pilares que garantiram a viabilidade matemática dos algoritmos testados.

Nas Redes Neuronais, ficou demonstrado que a escolha da topologia e do algoritmo de treino é determinante. O algoritmo **trainbr** (Regularização Bayesiana) revelou-se o mais robusto, eliminando o risco de overfitting e garantindo uma generalização perfeita para dados novos.

No CBR, validou-se que o sucesso do modelo está diretamente ligado à física do problema: dar prioridade a sensores como Vibração e Corrente foi fundamental para filtrar o ruído de variáveis de contexto.

Em conclusão, os resultados obtidos validam a eficácia das arquiteturas implementadas, demonstrando que tanto o modelo de Redes Neuronais como o sistema CBR são capazes de diagnosticar falhas elétricas e mecânicas com precisão absoluta quando os dados são corretamente tratados.